Solution Design Document – Document Intelligence Hub

Version History

V1.0 Phase 1- core pipeline

v1.5 phase-1.5 UI modularization and helper consolidation

Executive Summary

This project converts unstructured PDF collections into a locally hosted, searchable knowledge base. Phase 1 delivered the end-to-end pipeline for ingestion, text extraction, FTS5-based keyword search, analytics, and document library management. Phase 1.5 refined the internal architecture through UI modularization and helper consolidation, improving maintainability and presentation clarity without changing functional scope. The system is designed as a foundation for future AI-driven retrieval-augmented generation (RAG) capabilities.

Purpose of Solution Design Document

This document defines the design, scope, and technical approach for the document intelligence hub.

Project Background

Many academic, research, and professional workflows rely on PDFs, but these documents are often difficult to search and analyze as a collection. Users typically need to open files manually and search one-by-one, which is slow and inefficient. Document Intelligence Hub addresses this by creating a centralized searchable repository of extracted page-level text and metadata.

Project Team – Solo project

- Data Engineer: Built the SQLite FTS5 ingestion pipeline and metadata schema.
- Business Analyst: Defined the functional requirements for document search and analytics.

- Developer: Created the Streamlit UI and modular backend logic.

Point of Contact

Primary contact – Ritesh Reddy Koya

Business Requirements

- Functional Requirement: Ingest multiple PDF files and extract normalized text.
- Functional Requirement: Provide an interactive dashboard for document analytics (word counts, status, distribution).
- Search Requirement: Enable keyword-based search with real-time result snippets and page-jump capabilities

Scope

Included in Phase 1:

PDF upload / ingestion through Streamlit UI

- Text extraction and metadata extraction from PDFs
- SQLite-based storage (documents + pages tables)
- Duplicate detection using file hash
- Keyword search across page text
- SQLite FTS5-based search/ranking (BM25 scoring) for improved performance
- Search results table + Quick View preview
- Analytics dashboard (summary stats and top/recent docs)

Document library with:

- document details
- targeted page text preview
- PDF preview / download
- delete/remove document functionality

Ui improvements:
UI modularization — app.py refactored into a lightweight router; interface split into ingestion_page.py, search_page.py, library_page.py, analytics_page.py

- Helper consolidation — snippet generation and keyword highlighting centralized into utils.py and ui_helpers.py; duplicate logic removed

- Test coverage — tests/test_utils.py and tests/test_search.py added for core backend functions
- Code traceability improvements for maintainability and technical review

Out of Scope for Phase 1 (planned for later phases)

- Semantic search (embeddings)
- Vector database integration (ChromaDB/FAISS)
- RAG / question-answering over documents
- Citation-grounded answer generation
- User accounts / authentication
- Cloud deployment
- Production-grade logging/monitoring

Success Criteria

Phase 1 will be considered successful if the application can:

1. Ingest multiple PDFs successfully.
2. Prevent duplicate ingestion of previously indexed files.
3. Store document metadata and page text in SQLite.
4. Return keyword search results at page level with ranking.
5. Display analytics summary for ingested corpus.
6. Allow users to browse and preview documents in a library UI.
7. Allow deletion of documents and associated page records.
8. Demonstrate a stable end-to-end local workflow in Streamlit.
9. Codebase is modular.

Solution Overview

The system is organized into two layers. The backend (src/) contains all business logic — ingestion, database management, search, analytics, and utility helpers — with no dependency on the UI framework. The frontend (src/pages/) contains four dedicated page modules that call the backend and render output through Streamlit. app.py acts as a lightweight entry point and router, initializing logging, database setup, and session state before delegating to the appropriate page module.

UI Featues: ranked search results, quick preview, PDF viewing, and targeted page previews.

The design is intentionally modular so future phases can add:

- BM25 tuning / advanced search features
- semantic search with embeddings
- RAG-based Q&A with citations

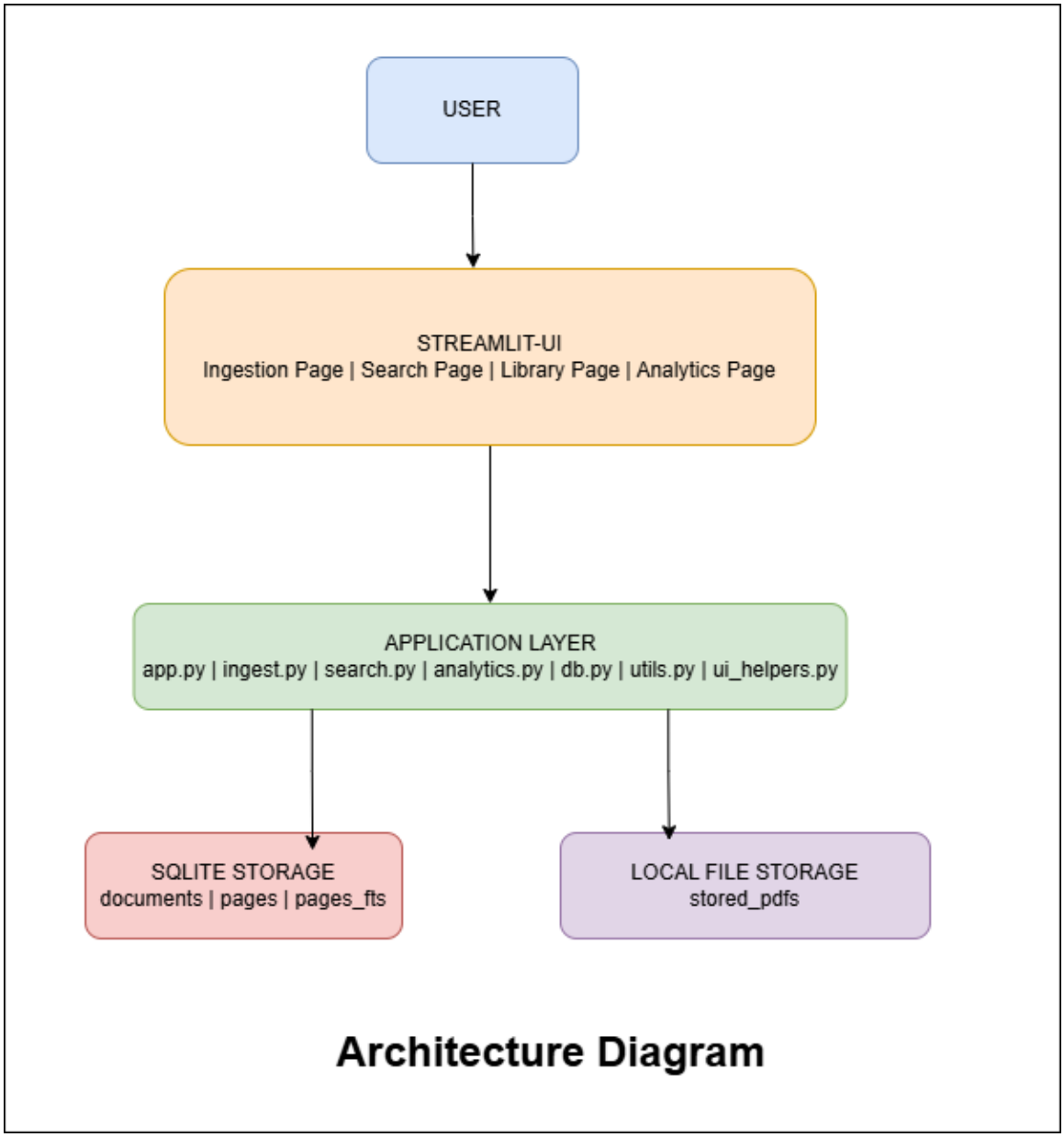
Implementation Timeline

Phase 1 – 1 Week

Technology Stack

- Frontend/UI: Streamlit
- Language: Python
- PDF Processing: PyMuPDF (fitz)
- Database: SQLite
- Search: SQLite FTS5 (BM25 scoring) + fallback search behavior
- Data Analysis / Tables: Pandas
- Local Storage: filesystem (stored_pdfs/)
- Development Environment: VS Code + Python virtual environment (Windows)

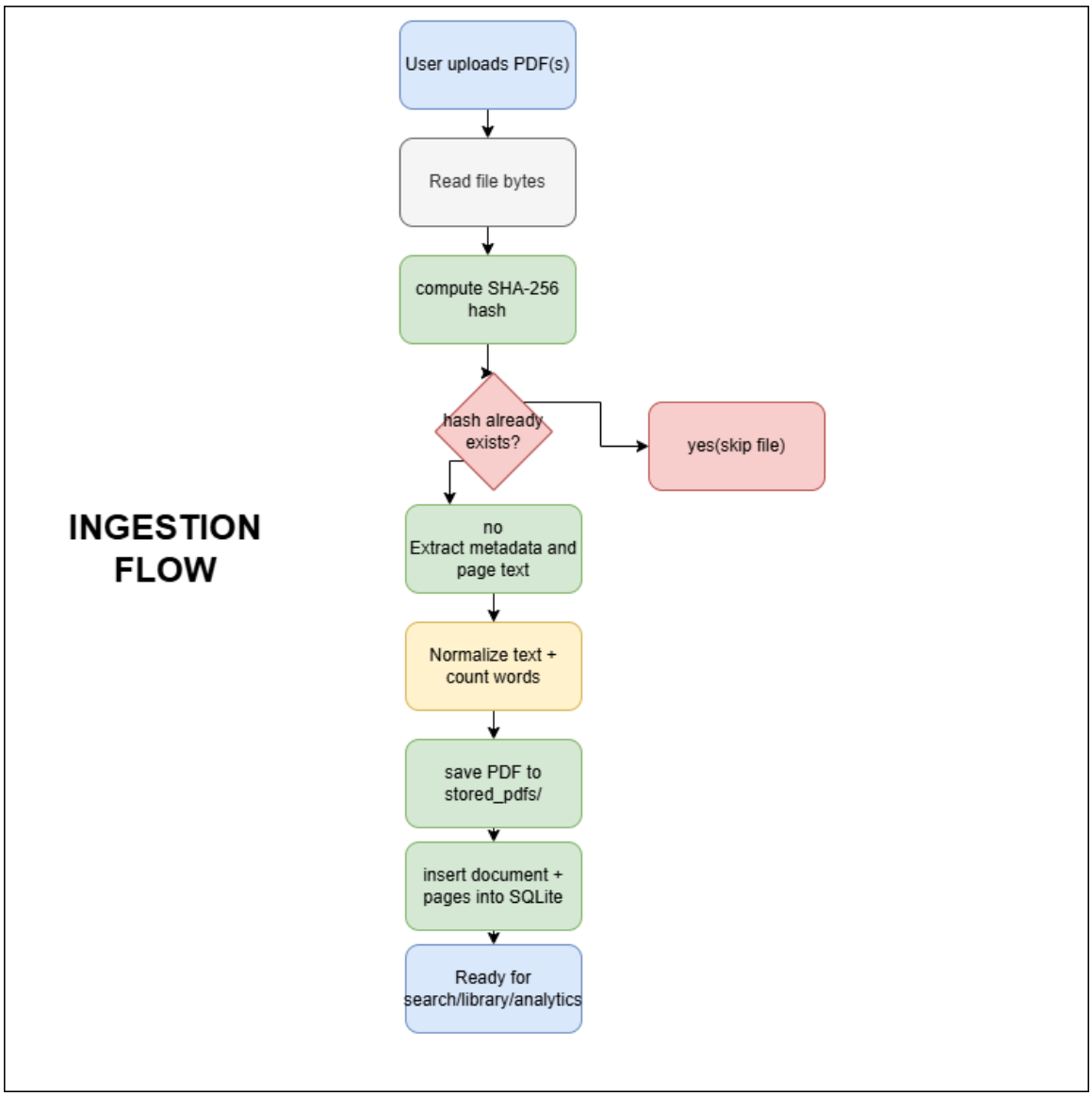
Architecture Diagram



The diagram below illustrates the two-layer architecture. The Streamlit UI layer (app.py + src/pages/) handles all user interaction and rendering. The backend layer (src/) handles ingestion, storage, search, and analytics with no UI dependencies. Data flows from PDF upload through ingestion and storage, then back up through search or analytics queries to the UI

Ingestion Flow

The ingestion flow below shows the step-by-step process from PDF upload to indexed storage. Key decision points include the duplicate hash check (which short-circuits re-ingestion) and the FTS5 trigger mechanism (which automatically keeps the search index in sync with the pages table without manual indexing steps).



Functional Requirements

- FR-1: System shall allow uploading of one or more PDF files via Streamlit UI.
- FR-2: System shall extract document metadata (file name, title, author, page count, file size).
- FR-3: System shall extract and store page-level text for each PDF.
- FR-4: System shall detect duplicate documents using file hash and skip re-ingestion.
- FR-5: System shall store document and page data in SQLite.
- FR-6: System shall support keyword search across page text.
- FR-7: System shall rank search results by relevance (FTS5/BM25 score).
- FR-8: System shall display search results with document/page metadata.
- FR-9: System shall provide quick preview of matched pages.
- FR-10: System shall provide a document library view for browsing ingested PDFs.
- FR-11: System shall provide targeted page text preview in library.
- FR-12: System shall support document deletion (including associated indexed pages).
- FR-13: System shall provide analytics summary of ingested documents.

Technical Requirements

- Python
- Streamlit application runs locally
- SQLite database for persistent local storage
- PDF parsing library capable of page-level text extraction (PyMuPDF)
- Local filesystem access for storing ingested PDFs
- Support for duplicate detection via file hashing
- SQLite FTS5 enabled for efficient keyword search and ranking
- Must handle common PDF ingestion errors without crashing the app
- UI must support local interactive use (ingest, search, analytics, library)

Assumptions and Considerations

- Users will run the application locally (Phase 1 local deployment only).
- PDF files are text-extractable (not all scanned/image-only PDFs may return usable text).
- Local machine has sufficient disk space for stored_pdfs and SQLite database.

Risks and Dependencies

- Data Quality: Handled via your normalization and error-logging logic in ingest.py.

- System Scalability: Addressed by moving from basic LIKE queries to FTS5 indexing.
- Dependency: Reliability of local storage for the stored_pdfs directory.